Degree Project in Technology

First Cycle, 15 hp

Robustness of Neural Networks on Optimized Synthetic Images

An Empirical Study of Robustness and Cross-Network Robustness Transferability

Samuel Fors

Adam Landbü

Abstract

Artificial neural networks have proven to be vulnerable even to small perturbations in their input. In order to prevent noise or maliciously handcrafted perturbations from eliciting undesirable behavior in the network, one must understand what makes a network more or less sensitive to these perturbations in its input. This is the study of robustness. Our main contribution is investigating the relationship between neural network classification confidence and robustness by suggesting models for generating images that maximize the output of neural networks in some different senses. We test if these generated images are more robust than normal images. We also investigate factors that contribute to making a generated image more robust, and conduct adjacent analysis. We show that images generated by what we call *selective output maximization* are indeed more robust compared to normal images. The extent to which such generated images are more robust varies greatly depending on the type of image class, and less so, but not insignificantly, on the architecture of the network that generated the images and the type of *selection function* that is used to generate the images. We also investigate a concept we call *robustness transferability*, a generated image's propensity to perform robustly on networks other than the one which generated it. We find that the generated images are also more robust on other networks that did not generate them compared to normal images, but less robust compared to the network that generated the images. We also propose an image generation method which simultaneously uses multiple neural networks to generate images. Tests hint at these so called *multi-models* resulting in slightly higher robustness transferability of their generated images.

Sammanfattning

Artificiella neurala nätverk har visat sig vara sårbara även för små störningar i sina indata. För att förhindra att brus eller avsiktligt konstruerade störningar orsakar oönskat beteende i nätverket krävs en förståelse för vad som gör ett nätverk mer eller mindre känsligt för sådana störningar av dess indata. Detta är vad som studeras inom robusthet. Vårt huvudsakliga bidrag är att undersöka sambandet mellan klassificeringssäkerheten hos neurala nätverks och deras robusthet genom att föreslå modeller för att generera bilder som maximerar neurala nätverks utdata i olika avseenden. Vi testar om dessa genererade bilder är mer robusta än normala bilder. Vi undersöker även faktorer som bidrar till att göra en genererad bild mer robust och genomför närliggande analys. Vi visar att bilder som genereras av det vi kallar *selektiv utdatamaximering* faktiskt är mer robusta jämfört med normala bilder. I vilken utsträckning sådana genererade bilder är mer robusta varierar kraftigt beroende på typen av bildklass, och mindre, men inte obetydligt, på arkitekturen hos nätverket som genererade bilderna och typen av *selektionsfunktion* som används för att generera bilderna. Vi undersöker även ett koncept som vi kallar *överförbarhet av robusthet*, det vill säga en genererad bilds benägenhet att uppvisa robusthet även på andra nätverk än det som genererade bilden. Vi finner att de genererade bilderna också är mer robusta i andra nätverk som inte genererade dem jämfört med vanliga bilder, men mindre robusta jämfört med nätverket som genererade bilderna. Vi föreslår dessutom en metod för bildgenerering som samtidigt använder flera neurala nätverk för att generera bilder. Tester antyder att dessa så kallade *multimodeller* ger upphov till något högre överförbarhet av robusthet hos de genererade bilderna.

Acknowledgments

We would like to thank our supervisor Johanna Skåntorp for her guidance, support, and feedback throughout the entire course of this project. Her encouragement and expertise has been greatly appreciated.

We would also like to thank our subject area coordinator Jan Kronqvist for his assistance during the course of the work.

Usage of Generative AI

Generative AI was used during the implementation of the code. Specifically, ChatGPT and Gemini helped debug the reason behind different types of errors that were encountered during programming. This helped us pinpoint the failures easier, which meant the relevant code documentation could be found and consulted quicker. These errors were then further investigated manually and fixed by the authors.

Additionally, Generative AI was also utilized during the writing of the report. ChatGPT was both used to locate grammatical errors and to point out when some text could be improved for the sake of readability. However, generative AI did not write any part of the report. All the text is entirely written by the authors.

Contents

1 Introduction **6**

2 Background: Neural Networks and Robustness **7**
2.1 Basic Neural Network Concepts . . . 7
2.2 Feedforward Networks . . . 7
2.3 Rectified Linear Unit . . . 8
2.4 Softmax Function . . . 8
2.5 Robustness . . . 9
2.5.1 Definitions . . . 10
2.5.2 More on Robustness . . . 11

3 Background: Verification Techniques **12**
3.1 Mixed-Integer Linear Programming Problem . . . 12
3.1.1 Big-M Formulation . . . 12
3.1.2 Bound Tightening . . . 13
3.2 Linear Approximation of ReLU . . . 13

4 Methods **15**
4.1 Dataset . . . 15
4.2 Neural Network Training . . . 15
4.3 Digits and Metrics . . . 15
4.4 Image Generation . . . 16
4.4.1 Selective Output Maximization . . . 16
4.4.2 Multi-Model . . . 17
4.5 Robustness analysis . . . 18
4.6 Technical Details . . . 18
4.7 Technical Specifications . . . 19

5 Results **20**
5.1 Robustness of Test Images . . . 20
5.1.1 Robustness across Digits . . . 20
5.1.2 Impact of Neural Network Size . . . 22
5.2 Accuracy of the Fast-Lin method . . . 22
5.3 Robustness of Generated Images . . . 24
5.4 Transferability of Generated Images . . . 27
5.4.1 Single-Model Transferability . . . 27
5.4.2 Multi-Model Transferability . . . 29

6 Discussion **30**

7 Conclusion **32**

A Appendix **35**
A.1 Alternative Image Generation Procedures . . . 35
A.2 The Most Robust Samples . . . 37
A.3 Gallery . . . 38
A.4 Accuracy of Trained Neural Networks . . . 39

1 Introduction

Neural networks are becoming increasingly prominent in a growing range of applications. One area in which neural networks have found especially notable success is within image classification, which is the primary focus of this report. However, despite their impressive accuracy, these models are often sensitive to small perturbations in their input [18, 8], leading to potentially unreliable results in the perturbation-prone real-world applications in which these models are deployed [13]. Furthermore, neural networks are vulnerable to *adversarial attacks*, inputs specially generated to elicit certain mistakes by the model [18, 8]. As neural networks are being adopted in more critical tasks, this unreliability has the potential to go from merely a nuisance to a life-or-death problem. More concretely, neural networks are already being used for things like phishing detection and malware detection. A malicious actor with sufficient knowledge could potentially craft an adversarial attack to bypass these critical systems [15]. Understanding the sensitivity to perturbations in the input, or robustness as it is often referred to, is therefore more important than ever. This report takes a mostly empirical approach in the study thereof, applying the theory to formulate experiments by which a better understanding of robustness may hopefully be achieved.

What does the robustness of a neural network depend on? There are, of course, many possible explanations to this, and we will investigate a few of them. This report mainly focuses on the relationship between robustness and a network's confidence in the classification of some original input, to use a mild anthropomorphism. Inspired by activation maximization [2, 16], we generate artificial images that aim to maximize the classification confidence of the network based on different objective functions. Our main contribution is generalizing activation maximization by suggesting alternative objective functions. We also generate images that simultaneously maximize the output of multiple different neural networks. The robustness of these artificially generated images is then compared against normal images in order to investigate any differences. Although our suggested methods for image generation are simple, the robustness of these specific different types of images has to the best of our knowledge not previously been studied. In this report, we hence collectively refer to these image generation methods as *selective output maximization*.

Structure of the Thesis

This report is structured as follows. We first introduce the necessary concepts and terminology. Neural networks and robustness are described in Section 2, and Section 3 explains how robustness guarantees can be computed by both exact and inexact approaches. Our methodology is described in Section 4, including the image generation procedure described in Subsection 4.4. The computed results are presented in Section 5. These results are discussed in Section 6, before giving our conclusions in Section 7.

2 Background: Neural Networks and Robustness

The robustness of a neural network refers to its ability to maintain stable predictions when the input is subject to perturbations. While many models achieve high accuracy on clean, unperturbed data, their performance can degrade significantly when the data is contaminated by noise or other forms of distortions [13]. This is particularly important in real-world applications, where the data is often imperfect. Models deployed in critical environments, such as self-driving cars, must therefore be able to handle variability.

More precisely, the robustness of a network is a per-sample metric. The task is to find the smallest perturbation, as measured by some norm, that causes the network to classify the perturbed input in some desired way. That desired way may be for the network to classify the input as some other specific class, which is referred to as *targeted attacks* [6]. It may also be, as this report mainly concerns, any class other than the correct one, which is referred to as *untargeted attacks* [6]. The robustness of the network can then be viewed as the average robustness across some set of samples, for example. Another reasonable heuristic is to say that the network's robustness is the minimum of all the minimum necessary perturbations. On the other hand, one might also conduct a robustness test of multiple networks on the same sample in order to get a sense of the robustness relating to that specific sample. That is, there is a sense in which a network is robust and there is a sense in which a sample is robust.

2.1 Basic Neural Network Concepts

Neural networks are a category of parametrized functions separable into layers of simple functions. There are many types of neural networks, so further restricting the definition risks excluding certain kinds. This report pertains exclusively to feed-forward classification neural networks, more specifically fully-connected networks. These networks are sometimes also called dense neural networks because of their structure. Their architecture consists of layers of linear functions. Compositions of exclusively linear layers would restrict the network to be a linear function itself, so to combat this non-linear layers, or activation functions as they are commonly referred to as, are introduced between the linear functions.

One of the most common activation functions is the piecewise linear *rectified linear unit* (ReLU). The usage of ReLU is well motivated for purposes such as computational efficiency, countering the vanishing gradient problem, and other practical reasons [1]. It is therefore a very common choice in modern machine learning [10]. In fact, choosing a piecewise linear activation function also results in the nice property that the neural network as a whole becomes a piecewise linear function. We later show how a network of this type can be analyzed through the lens of mixed-integer linear programming [19].

2.2 Feedforward Networks

Feedforward networks consist of multiple layers composed together. Let $L + 1 \in \mathbb{N}$ denote the number of layers of the feedforward neural network, numbered by $0, \ldots, L$. Layer $\ell = 0$ is the input layer, and layer $\ell = L$ is the output layer. Also, let $n_\ell \in \mathbb{N}$ denote the number of neurons in layer $\ell = 0, \ldots, L$.

First, outputs from the previous layer are transformed by a so-called linear function. The pre-

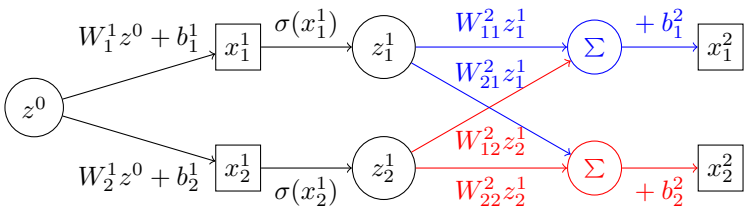

Figure 1: Example of a small neural network.

activation values $x^\ell \in \mathbb{R}^{n_\ell}$ for the neurons of layer $\ell = 1, \ldots, L$ are given by

$$x^\ell = W^\ell z^{\ell-1} + b^\ell, \tag{1}$$

where $W^\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ are the weights and $b^\ell \in \mathbb{R}^{n_\ell}$ are the biases.

These new variables are then fed into a non-linear function called the activation function. The post-activation values $z^\ell \in \mathbb{R}^{n_\ell}$ for the nodes of layer $\ell \in 1, \ldots, L-1$ are given by

$$z^\ell = \sigma^\ell(x^\ell), \tag{2}$$

where $\sigma^\ell : \mathbb{R}^{n_\ell} \to \mathbb{R}^{n_\ell}$ denotes the activation function for layer ℓ.

Since z^0 denotes the input to the neural network, this term does not actually have a corresponding pre-activation value. Similarly, we do not apply the activation function to the last layer, which means that there is no post-activation value z^L corresponding to the pre-activation value x^L.

An example of a neural network is shown in Figure 1. The input z^0 is transformed layer by layer into the outputs x_1^2 and x_2^2.

2.3 Rectified Linear Unit

While there exist many different types of activation functions, we consider networks in which every activation function is the rectified linear unit. Given an input vector $x \in \mathbb{R}^{n_\ell}$, the rectified linear unit $\text{ReLU} : \mathbb{R}^{n_\ell} \to \mathbb{R}^{n_\ell}$, most often simply called the ReLU function, defined by

$$\text{ReLU}(x) = \max\{x, 0\}, \tag{3}$$

is an element-wise operation on the input [3, 20]. The ReLU function on a single variable $x \in \mathbb{R}$ is shown in Figure 2. Importantly, notice that the ReLU function is piecewise linear. It thus follows that the entire network is piecewise linear.

After activation by the ReLU function, neurons are either zero-valued or have a positive value. If the neuron is zero-valued it is called *inactive*. If the neuron has a positive value it is called *active*.

2.4 Softmax Function

A very important aspect is how the neural network classifies inputs. Usually, the softmax function is used [7]. Softmax is a function very often applied to the output layer of classification networks

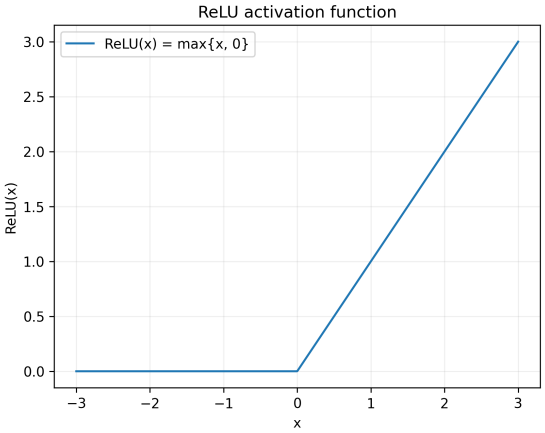

Figure 2: The ReLU activation function for inputs $x \in [-3, 3]$.

which transforms the output into a probability-like discrete distribution [7]. Let $x^L \in \mathbb{R}^{n_L}$ be the output of the neural network. These values are usually called logits. The function softmax : $\mathbb{R}^{n_L} \to [0,1]^{n_L}$ normalizes the logits into probabilities by the non-linear transformation

$$\text{softmax}(x^L)_i = \frac{e^{x_i^L}}{\sum_{j=1}^{n_L} e^{x_j^L}}, \tag{4}$$

for each output class $i \in 1, \ldots, n_L$. The network then classifies the input as the class with the highest output probability. Letting $y(x)$ denote the classification of the input x by the neural network $f(\cdot)$, the classification is then

$$y(x) = \underset{i \in 1,\ldots,n_L}{\arg\max}\, \text{softmax}(f(x)_i). \tag{5}$$

Notice that the softmax function is not piecewise linear. Including this function therefore destroys the piecewise linearity of the ReLU network. Thankfully, the softmax activation can be ignored for robustness analysis. This is because the softmax function is strictly increasing with respect to each logit. The ordering of the logits is therefore preserved after softmax normalization. In particular, the predicted class is determined by

$$\underset{i \in 1,\ldots,n_L}{\arg\max}\, \text{softmax}(f(x)_i) = \underset{i \in 1,\ldots,n_L}{\arg\max}\, f(x)_i. \tag{6}$$

We will be using softmax while training the networks, but all the robustness analysis will be carried out by using the logits to preserve the piecewise linearity of the problem. As shown, this does not affect the results.

2.5 Robustness

Robustness is a measure of a neural network's resistance to perturbations, quite possibly adversarial ones, in the input [8, 18]. There are multiple more precise adjacent definitions used in the literature. Below we introduce those relevant to this report as well as provide some context to alternatives.

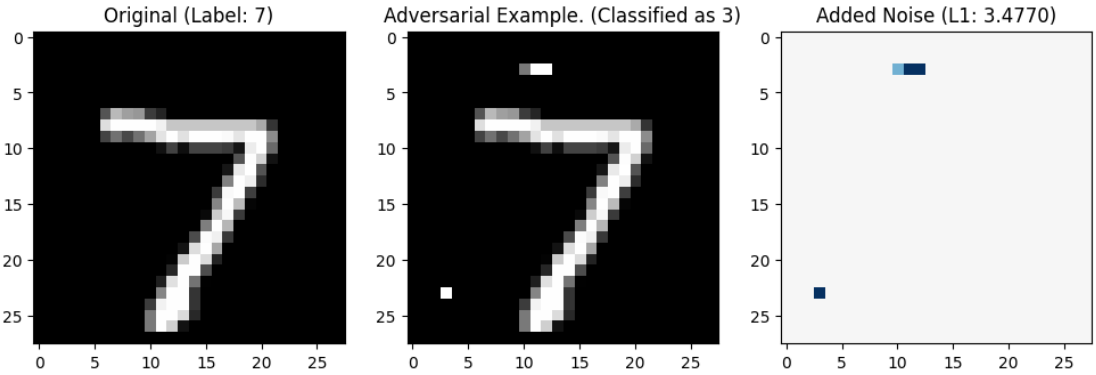

Figure 3: Example of an adversarial image. The original image on the left was correctly identified as a seven by the neural network. However, after this image is perturbed, the image is now incorrectly identified as a three. The image to the right shows the added noise and its ℓ_1 norm.

2.5.1 Definitions

Let $X \subseteq \mathbb{R}^{n_0}$ be the input space of the neural network $f : X \to Y$. Again, let $y(x)$ denote the classification of the input x by the neural network f, that is

$$y(x) = \arg\max_j f_j(x). \tag{7}$$

Given an input $x_0 \in X$ and a perturbation $\varepsilon \in \mathbb{R}_{\geq 0}$ in ℓ_p-norm, will the network still classify the input the same? This amounts to verifying the truth of the statement

$$x \in B_p(x_0, \varepsilon) \implies y(x) = y(x_0). \tag{8}$$

Here $B_p(x_0, \varepsilon)$ denotes the ε-ball centered at the point x_0 given an ℓ_p norm, precisely defined by

$$B_p(x_0, \varepsilon) = \{x \in X : \|x - x_0\|_p \leq \varepsilon\}. \tag{9}$$

Assume that the network classifies the unperturbed input as $y(x_0) = i$. Given some other classification $j \neq i$, we may check if any perturbed input $x \in B_p(x_0, \epsilon)$ causes the network to predict a higher probability of the sample being of class j than i. This is done by solving the optimization problem

$$\begin{aligned} \underset{x \in \mathbb{R}^{n_0}}{\text{minimize}} \quad & f_i(x) - f_j(x) \\ \text{subject to} \quad & x \in B_p(x_0, \varepsilon). \end{aligned} \tag{10}$$

If the optimal value is negative, there exists an adversarial input $x \in B_p(x_0, \varepsilon)$ that the network classifies as more likely being of class j rather than of class i. A solution with negative objective value is called an adversarial input [8, 18]. Such an input is close to the original input but classified as more likely being of class j than i. An example of an adversarial image is shown in Figure 3. It is important to note that the network may still classify a perturbed input $x \in B_p(x_0, \varepsilon)$ as another classification $y(x) = k$, where $i \neq k$ and $j \neq k$. To guarantee that the perturbed input is never classified as anything other than the unperturbed class i, one would have to solve the above minimization problem 10 for all other classes $j \neq i$.

We have now seen how robustness can be verified for a constant $\varepsilon \in \mathbb{R}_{\geq 0}$. It is of high interest to find the maximal possible perturbation ε such that the network is still robust. This may be formulated as the optimization problem

$$\begin{aligned} \varepsilon = \underset{x \in \mathbb{R}^{n_0}}{\text{minimize}} \quad & \|x - x_0\|_p \\ \text{subject to} \quad & y(x) \neq y(x_0). \end{aligned} \tag{11}$$

This guarantees that every interior perturbation in the ε-ball is classified the same by the neural network.

Since fully-connected feedforward ReLU neural networks are piecewise linear, the above optimization problems 10 and 11 can be formulated as mixed-integer linear programming problems when the input constraints corresponding to the ε-ball are piecewise linear. Notably this is true for the ℓ_1 and ℓ_∞ norms.

As mentioned previously, the neural network $f(\cdot)$ may or may not have a softmax activation on the last layer's neurons. However, since softmax is a strictly increasing function, none of the robustness analysis changes.

2.5.2 More on Robustness

Although all definitions of robustness, the ones above included, try to capture an idea of perturbation sensitivity in different senses, they differ in key ways. We discuss alternatives mainly for the sake of context.

As briefly mentioned, one fundamental difference in approach is per-input robustness versus per-dataset robustness. Many robustness evaluations search for the largest neighborhood within which the network's classifications remain unchanged across a dataset [15]. Others, like (10) and (11) above, search for such a neighborhood on a per-sample basis. Related are the concepts of a global perturbation and a local perturbation, to either apply the same perturbations across all samples in the dataset and test how the classifications change or to do so with per-sample perturbations. In our experiments we will only use per-sample techniques and tests but we will average the results over multiple tested samples. Note that this is different from testing global perturbations.

To talk about the size of a perturbation, it of course needs to be in regard to some norm. As mentioned, the norms ℓ_1 and ℓ_∞ are the only norms immediately accessible through linear constraints and are therefore commonly used when solving robustness problems formulated as mixed-integer linear programming. Other approaches, such as Fast-Lin [20], are however not so tightly confined and may without issue use other ℓ_p norms. In practice this is commonly the ℓ_2 norm [15].

3 Background: Verification Techniques

As hinted at, finding the robustness of an input amounts to solving an optimization problem. In this section we show how mixed-integer linear programming can be used. However, since the mixed-integer linear programming approach is computationally infeasible for larger networks [3], we also describe how one can obtain non-trivial lower bounds of the robustness using a significantly faster method called Fast-Lin [20].

3.1 Mixed-Integer Linear Programming Problem

Mixed-integer linear programming (MILP) is a field within mathematical optimization adjacent to linear programming. Linear programming pertains to optimization problems which can be formulated with linear constraints and a linear objective function [21]. The mixed-integer part of the MILP stems from it being linear programming with some variables restricted to be integer-valued, in contrast to the more general real numbers used in linear programming [21]. MILP is an incredibly expressive framework but suffers from being comparatively computationally heavy. More precisely, solving a MILP problem is NP-hard [17]. Informally, this means that there is unlikely to exist an algorithm that solves the problem in polynomial time [12]. For large neural networks, solving the robustness MILP problems becomes infeasible for most inputs. However, for small networks this is still manageable [3]. Most components of fully-connected neural networks are affine, which allows them to be formulated as sets of linear constraints. However, the expressiveness of the networks is heavily dependent on the non-linear activation functions between the layers. Exclusively using the ReLU activation function makes the networks piecewise linear functions, which allows us to express these as MILP constraints [19].

3.1.1 Big-M Formulation

The piecewise linear nature of the feedforward ReLU networks allows us to model their behavior as MILP constraints. A common method for formulating the ReLU logic as MILP constraints is the big-M formulation [10], where the logic of the post-activation value z_j^ℓ is given by

$$\begin{aligned} z_j^\ell &\geq W_{j,:}^\ell z^{\ell-1} + b_j^\ell, \\ z_j^\ell &\geq 0, \\ z_j^\ell &\leq (W_{j,:}^\ell z^{\ell-1} + b_j^\ell) - M_{j,-}^\ell (1 - \lambda_j^\ell), \\ z_j^\ell &\leq M_{j,+}^\ell \lambda_j^\ell, \\ z^{\ell-1} &\in [L^{\ell-1}, U^{\ell-1}], \\ \lambda_j^\ell &\in \{0, 1\}, \end{aligned} \tag{12}$$

where $L^{\ell-1}$ and $U^{\ell-1}$ are lower and upper bounds of the neurons in layer $\ell - 1$. The binary variable λ_j^ℓ is zero if the j-th neuron of layer ℓ is inactive and one if it is active. Additionally, the values $M_{j,+}^\ell$ and $M_{j,-}^\ell$ must satisfy the constraints

$$\begin{aligned} M_{j,-}^\ell &\leq \min_{z^{\ell-1} \in \mathcal{Z}^{\ell-1}} \{W_{j,:}^\ell z^{\ell-1} + b_j^\ell\}, \\ M_{j,+}^\ell &\geq \max_{z^{\ell-1} \in \mathcal{Z}^{\ell-1}} \{W_{j,:}^\ell z^{\ell-1} + b_j^\ell\}, \end{aligned} \tag{13}$$

where $\mathcal{Z}^{\ell-1}$ is the set of feasible post-activation values from the previous layer.

3.1.2 Bound Tightening

The solution time of the MILP problem depends heavily on the bounds $M^{\ell}_{j,+}$ and $M^{\ell}_{j,-}$ [10]. It is therefore important to be able to obtain sufficiently tight bounds.

If we know, which for most practical applications we do, the bounds of the input we can use these to calculate the bounds on the variables or, often more feasibly, at least obtain an estimate thereof. This can be computed exactly for each neuron by formulating and solving two optimization problems, one for its lower bound and one for its upper bound. For the bounds on each neuron, this only requires solving an optimization problem of the constraints corresponding to the neurons that appear in earlier layers than the current neuron. Naturally, solving a MILP problem for each neuron quickly becomes computationally burdensome, especially as the number of layers grow.

Alternatively, we can use the the bounds on the input to calculate bounds on the first layer and then iteratively calculate the next layer's bounds with the previous. Note that the bounds calculated through this bound propagation will be valid, but may not be tight. The layers forward pass is just not simple enough to allow for perfectly tight bound propagation. A simple method of bound propagation is using interval arithmetic. It is also possible to calculate bounds by the Fast-Lin method, which we will present next.

3.2 Linear Approximation of ReLU

Fast-Lin is a method for computing *certified lower bounds* on robustness of feedforward ReLU networks [20]. A certified lower bound is simply a non-trivial perturbation radius $\varepsilon \in \mathbb{R}_{\geq 0}$ such that the neural network does not change the classification of a perturbed input $x \in B_p(x_0, \varepsilon)$ [20]. Notice that this is different from the maximum allowed ε as formulated in equation (11). The certified lower bound is simply a lower bound of this value. An example illustrating the difference between exact robustness and certified lower bounds is shown in Figure 4.

The Fast-Lin method achieves this by propagating linear upper bounds and lower bounds through each layer and replacing the non-linear activations with linear relaxations. These bounds are then used to guarantee that the output corresponding to the predicted class remains larger than the output corresponding to any other class within the allowed perturbation.

Fast-Lin obtains these certified lower bounds efficiently compared to the computational effort required to solve the mixed-integer linear programming formulation for large problems. Further implementation details and derivations are given in [20].

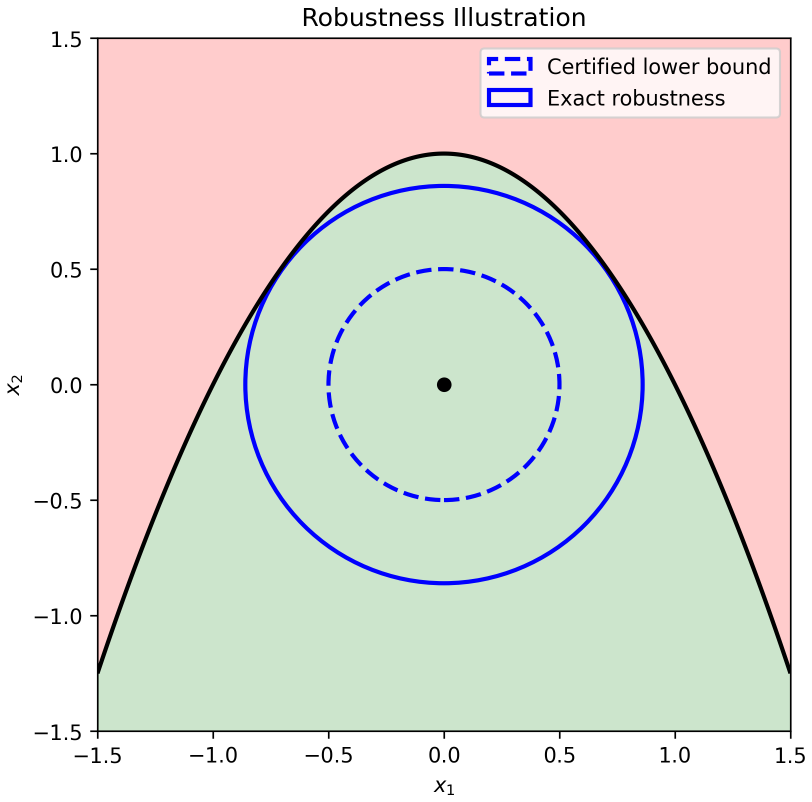

Figure 4: The figure illustrates the robustness of the input $x_0 = (0, 0)$ for a two-variable neural network. The network either classifies the points as green or red. Two ε-balls corresponding to the exact robustness and a certified robustness bound are shown. Note that the ℓ_2 norm is used in this example.

4 Methods

Naturally, one requires both a dataset and different types of neural networks to investigate robustness. First, the dataset and training procedures are described in Subsections 4.1 and 4.2. After that, Subsection 4.3 explains which digits that are investigated and our choice of norm. We suggest models for generating images from neural networks that aim to maximize their classification confidence. We call this image generation procedure *selective output maximization*. The image generation procedure is further introduced, defined, and discussed in Subsection 4.4. The robustness analysis, that is how these images are evaluated, is then explained in Subsection 4.5. Lastly, the technical details and specifications of our computations are provided in Subsections 4.6 and 4.7.

4.1 Dataset

The networks are trained on the MNIST dataset. The MNIST dataset is an industry standard dataset in machine learning. The dataset consists of 70 000 28x28-pixel grayscale images of handwritten digits. It is split into 60 000 images used for model training and 10 000 images set aside for testing purposes [14].

Although there is nothing inherently visual to robustness testing or selective output maximization, it is desirable to work with image classification since it produces interpretable results and very naturally lends itself to visualizations. Among other things, possible interesting patterns can then be identified more easily.

4.2 Neural Network Training

The robustness analysis is performed using multiple different fully-connected neural networks. These networks are designed to systematically vary the two structural properties: width (number of neurons per layer) and depth (number of layers). This design allows us to investigate the effect of network depth and width on robustness. Importantly, each network uses only ReLU activations. The trained neural networks and their classification accuracies for digits zero (0), one (1), and three (3) are presented in Table 1 in Appendix A.4.

Each network is trained for two epochs and uses cross-entropy as the loss function. The model parameters are tuned via the Adam optimizer [11] with a learning rate of 0.001. For each training epoch the model iterates over the entire dataset in small batches. For each batch, a forward pass computes predictions, the loss is evaluated against the true label of the images, and gradients are computed via backpropagation. The optimizer then updates the model weights accordingly.

4.3 Digits and Metrics

To investigate whether different classifications in general have different robustness, tests are conducted on three different digits (classes). We limit our analysis to the digits zero, one, and three (0,1,3). These digits are chosen because they are visually distinct, and can hopefully give rise to interesting differences in robustness.

The robustness analysis is performed using the ℓ_1 norm. The ℓ_1 norm can be implemented linearly which leads to faster computations than for example the ℓ_2 norm (which would require quadratic constraints).

In theory, robustness only concerns itself with a network's change in classification [8]. A neural network may therefore be considered robust even on images it misclassifies. In practice, however, classifications should be both correct and robust. As such, we only consider the robustness of inputs which are classified correctly by the network.

4.4 Image Generation

The main contribution of this report is to study how maximizing network confidence via generation of artificial images affects robustness. We suggest models for generating artificial images from neural networks that aim to maximize their classification confidence. We call these methods for generating artificial images *selective output maximization*. We also suggest how one can generate such artificial images by optimizing multiple neural networks simultaneously.

4.4.1 Selective Output Maximization

The *selective output maximization* of a neural network maximizes some given *selection function* on the output in order to generate an artificial image intended to maximize the classification confidence of the network on this image. Although this definition is very wide, the intended aim of selective output maximization is to quantify the network's classification confidence without relying on the softmax function. Selective output maximization takes inspiration from previous work such as [2, 16]. In particular, the first presented type of selection function is especially similar to activation maximization [2].

Given the output $f(x) \in \mathbb{R}^{n_L}$ of a neural network, we can consider functions $g : \mathbb{R}^{n_L} \to \mathbb{R}$ which select for (desirable) qualities in the output. We call such functions *selection functions*. A simple example is what we are calling the *independent* selection function. For a target class i, it simply selects the i-th element of the output vector, for example,

$$g_{\text{independent}}(f(x)) := f_i(x). \tag{14}$$

For any selection function $g(\cdot)$ and input space X, the *selective output maximization* is simply given by

$$\max_{x \in X} g(f(x)), \tag{15}$$

and the corresponding input $x^* \in X$, which maximizes $g(\cdot)$, is given by

$$x^* = \arg\max_{x \in X} g(f(x)). \tag{16}$$

Finding the input produced by selective output maximization in the *independent* sense therefore amounts to finding the input which maximizes the value of the node associated with the target class. This type of selective output maximization is similar to previous work where the single output neuron i was maximized in some way [2, 16].

Although simplicity is desirable, one potential problem is that the *independent* selection function does not take the other output neurons into account: an input x^* which maximizes the output of the i-th neuron does not at all guarantee that $f_i(x^*) > f_j(x^*)$ for all other output neurons.

For this reason we also introduce, what we call, the *mean relative* selection function, defined as

$$g_{\text{mean\_relative}}(f(x)) := f_i(x) - \frac{1}{n_L} \sum_j f_j(x). \tag{17}$$

Figure 5: Artificial zero generated from the 2x40 network using the independent selection function.

Here, the values of the non-target-class neurons are taken into account, while retaining simplicity. However, there are still easily imagined theoretical network outputs which could make the *mean relative* selection function a poor metric of network confidence.

To try to account for this we introduce a third selection function, which we call *second place relative*. The idea is to maximize the difference between the target-class neuron's value and the second largest output value. The function is given by

$$g_{\text{second\_place\_relative}}(f(x)) := f_i(x) - \max_{j \neq i} f_j(x). \tag{18}$$

An example of an image generated by selective output maximization is shown in Figure 5. The image contains seemingly random pixel-values near the image borders. This is not particularly unexpected, since the most important pixels for classification are the central ones.

4.4.2 Multi-Model

The functions above are designed to select for factors that, in a sense, reflect the network's confidence in its classification. The idea is that images which are classified with high confidence could also correspond to robust images. Not only is the choice of metric ambiguous, but we can also consider the confidence of more than one network when performing selective output maximization.

Given m neural networks $f_1, \ldots, f_m : X \to \mathbb{R}^{n_L}$ and a selection function $g : \mathbb{R}^{n_L} \to \mathbb{R}$, we can find the input $x^* \in X$ which maximizes the *multi-model* selective output by solving

$$x^* = \arg\max_{x \in X} \sum_{i=1}^{m} g(f_i(x)). \tag{19}$$

In this paper we only consider pairs of networks, that is $m = 2$. When $m = 1$ we say that the image has been generated by a *single-model*, and when $m \geq 2$ we say that the image has been generated by a *multi-model*.

4.5 Robustness analysis

First, the robustness of the MNIST test set is analyzed in order to develop a robustness baseline. Specifically, the robustness of each investigated digit (zero, one, three) is computed. Using the Fast-Lin method, the robustness of 100 randomly selected images of each digit is computed for every network. Using the exact method (MILP), the robustness of 5 images of each digit is computed for the networks. For the sake of better analysis, these images are randomly selected from the images that were also computed by Fast-Lin. We compare differences between the exact method (MILP) and inexact method (Fast-Lin) for these images. Additionally, general trends on how network architecture impacts robustness are investigated. This is simply accomplished by plotting the robustness of the networks against the number of neurons of the networks and the depth (number of hidden layers) of the networks.

After having established the robustness of test images, artificial images are generated for the networks using selective output maximization. We attempt to generate an image for every possible pair of digit (0, 1, 3) and selection function (independent, mean relative, second place relative). The robustness of these images is calculated using the Fast-Lin method for every network. Additionally, 100 robustness results are computed using the exact method (MILP). First, in order to properly justify the usage of Fast-Lin for the robustness analysis on the generated images, it is important to know that the Fast-Lin method still works well on these generated images. Therefore, the accuracy of Fast-Lin on test images is compared to the accuracy of Fast-Lin on generated images. This is accomplished by dividing the exact robustness results by their corresponding certified lower bounds, and analyzing the resulting distribution.

After finding how well Fast-Lin performs on the generated images, we investigate whether the generated images are more robust when compared to images from the test set. We determine if the type of digit and the choice of selection function affects the robustness.

Lastly, we examine whether these generated images are highly specific to the neural network that generated them, or if they generalize well across other neural networks (that did not generate them). This includes analyzing if the architecture of the network that generates artificial images has an effect on the robustness of the resulting images when tested on other networks. We also attempt to generate images using 100 different randomly selected multi-models. The transferability of the multi-model images is then analyzed in order to investigate whether these multi-models produce more transferable images than the single-models.

4.6 Technical Details

For computational reasons, the computations relating to the exact methods are timed out if they have not completed within five minutes. Specifically, these computations are the calculations of the exact robustness for images, and the generation of artificial images. The computations that are terminated are simply ignored and are not considered for any results. We also choose to only perform these exact robustness computations on networks with less than 100 hidden neurons in total, in order to avoid unnecessarily wasting time by timing out lots of models that are infeasible for these exact methods regardless [3].

Additionally, to make the results as realistic as possible, we only choose to investigate the subset of networks that classify each of the chosen digits (zero, one, three) with more than 90% accuracy. This is done in order to avoid that small networks with bad accuracy could possibly yield unreasonably different results compared to models that are more representative of real-world applications.

The specific images used for testing robustness are randomly selected. Since the number of possible neural network pairs is very large, the multi-model generation also randomly selects neural network pairs that individually have at most 100 hidden neurons. We consider 100 such different pairs of neural networks when generating multi-model images.

4.7 Technical Specifications

The implementation is done in Python 3.11.5 [4]. The MILPs are solved using GurobiPy (version 13.0.0) [9]. Note that the implementation requires a non-default Gurobi license. The networks are trained using PyTorch (version 2.10.0) [5]. For further details about the implementation, please visit `https://github.com/landbu/Kex`. The computer on which the experiments are conducted runs on an AMD Ryzen 5 5600 (6 cores, 12 threads, 3.5 GHz) CPU.

5 Results

This section presents robustness results for both MNIST test images and artificially generated images. We first analyze the robustness of the test images. The accuracy of the Fast-Lin method on the generated images is compared to the accuracy of the method on the test images. We then present the robustness of the generated images and investigate how this robustness varies across digit classes and selection functions. Finally, we analyze how well the robustness of the generated images transfers across networks that did not generate the images. This is done both for images generated by single-models and for images generated by multi-models.

Some additional robustness results that fall outside the scope of this report are presented in Appendix A.

5.1 Robustness of Test Images

First, the robustness analysis of the test images is presented.

5.1.1 Robustness across Digits

The robustness of the networks on the MNIST test images is shown in Figures 6, 7, 8, and 9, which displays both the results of the exact MILP-computations as well as the bounds obtained from the Fast-Lin method.

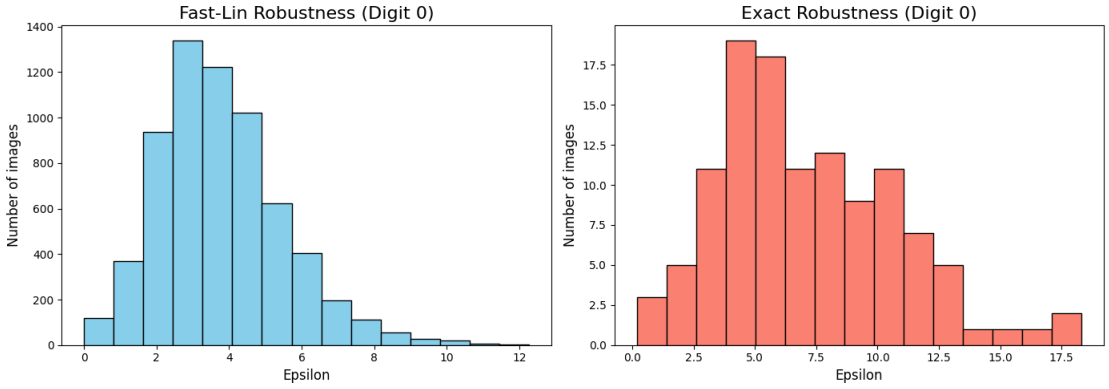

Figure 6: Robustness for all networks on a subset of images correctly classified as zero. Computed both by Fast-Lin (left) and by MILP (right).

From Figures 6, 7, 8, and 9 we see that there is a noticeable difference in the general robustness of each digit. On average, the zeros are more robust than the ones and threes. This is consistent between the exact robustness calculated by MILP and the inexact robustness calculated by Fast-Lin.

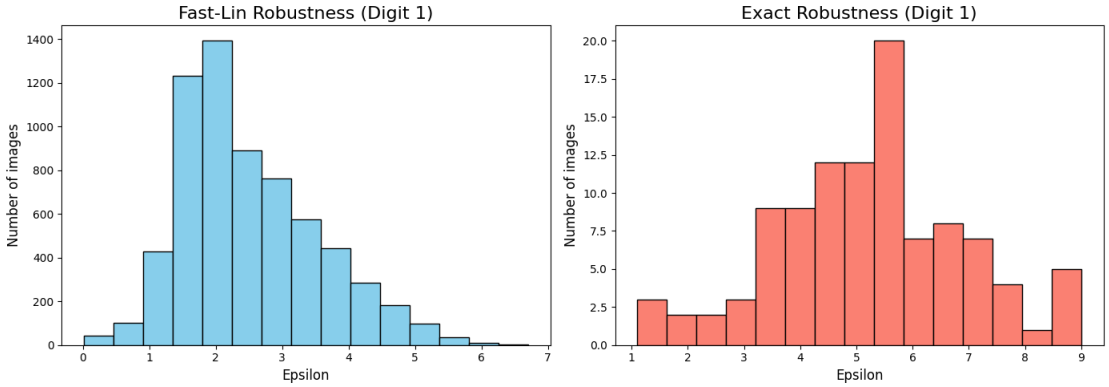

Figure 7: Robustness for all networks on a subset of images correctly classified as one. Computed both by Fast-Lin (left) and by MILP (right).

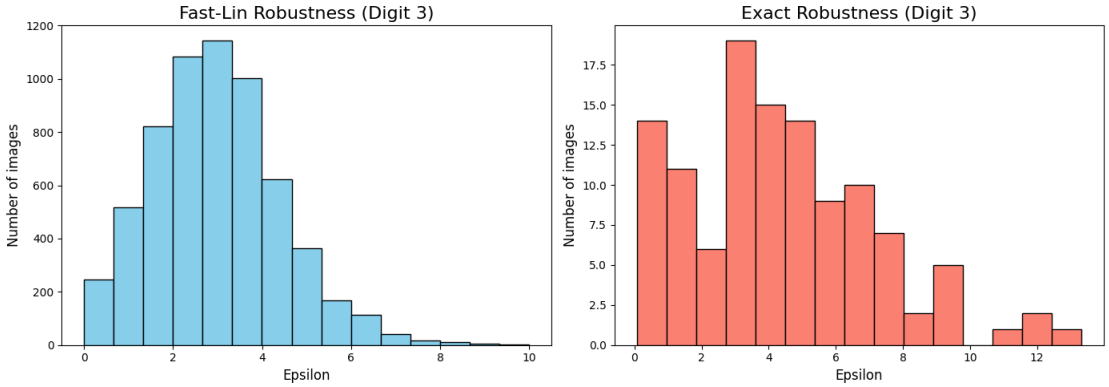

Figure 8: Robustness for all networks on a subset of images correctly classified as three. Computed both by Fast-Lin (left) and by MILP (right).

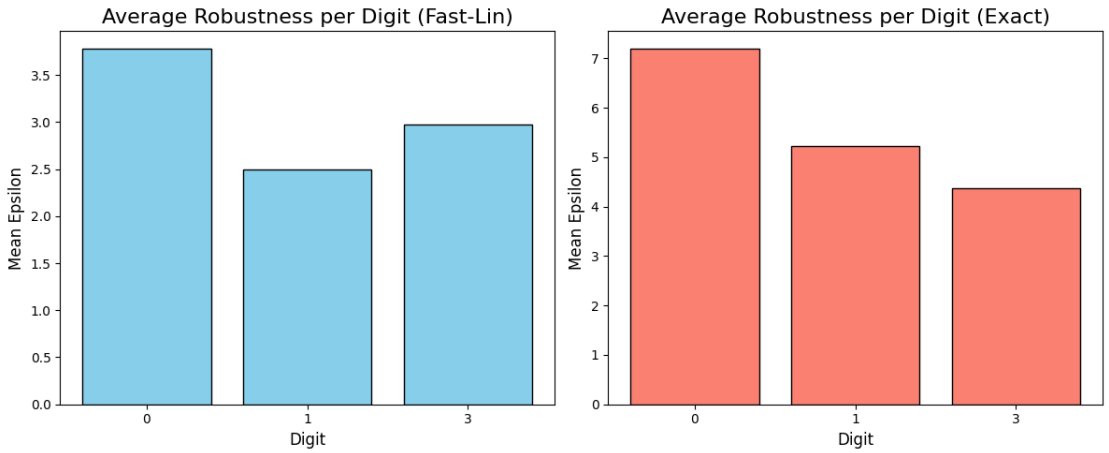

Figure 9: Average robustness for all networks on a subset of images correctly classified as either zero, one, or three. Computed both by Fast-Lin (left) and by MILP (right).

5.1.2 Impact of Neural Network Size

The impact of the size of a neural network on its robustness is shown in Figures 10 and 11.

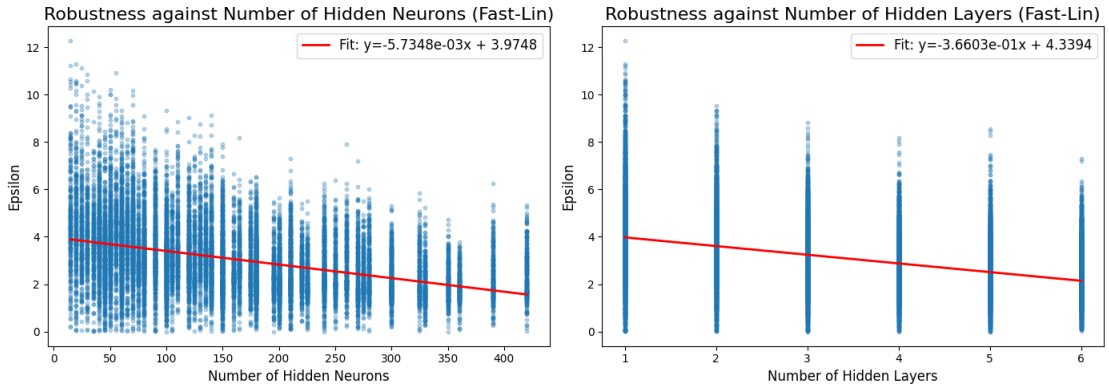

Figure 10: Certified lower robustness bounds calculated by Fast-Lin plotted against the total amount of neurons and the number of hidden layers of the networks.

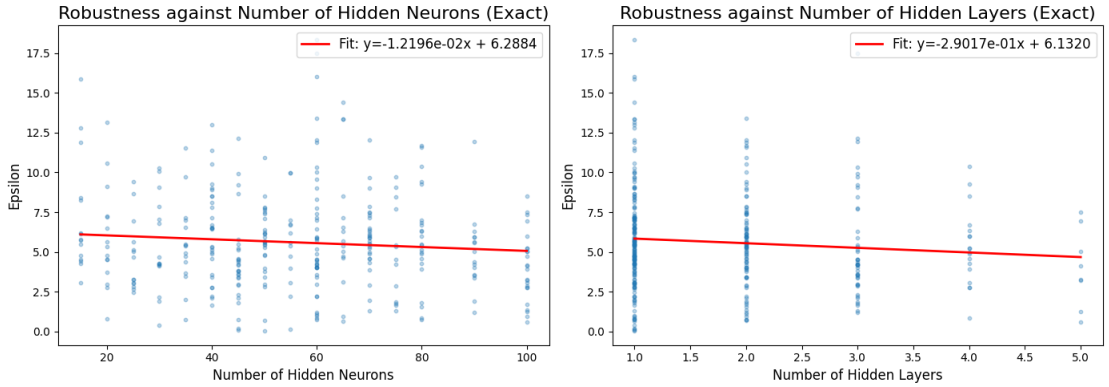

Figure 11: Exact robustness calculated by MILP plotted against the total amount of neurons and the number of hidden layers of the networks.

Figures 10 and 11 indicate that the robustness of a network in general decreases somewhat as the number of hidden neurons and the number of hidden layers grow.

5.2 Accuracy of the Fast-Lin method

Figure 12 shows how much the certified lower bound differs from the exact robustness results. From Figure 12 we find that most of the exact robustness values were less than double the inexact robustness bounds.

Figure 13 illustrates how much the certified lower bound differs from the exact robustness results for the generated images. From Figure 13 we find that most of the exact robustness values of the generated images were less than triple the inexact robustness bounds. When compared

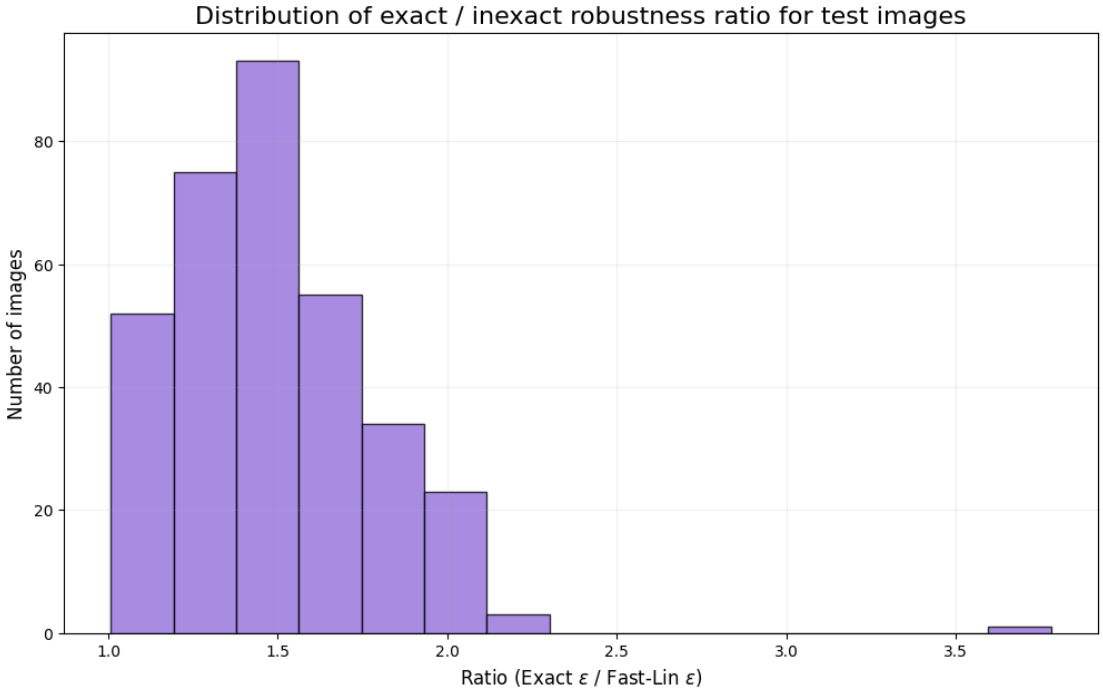

Figure 12: Exact robustness calculated by the MILP divided by certified lower bound calculated by the Fast-Lin method. Only considers MNIST test images.

to the MNIST test images in Figure 12, Fast-Lin appears to be somewhat less consistent on the generated images. However, crucially, the sample correlation between the robustness results obtained by the exact and the inexact methods is 0.945 for the test images and the corresponding sample correlation for the generated images is 0.965, both of which are very high. Although not a perfect proxy, these results suggest that Fast-Lin still provides a useful approximation of robustness for the purposes of this study. Thereby, for computational reasons, the remaining results are calculated using only the Fast-Lin method.

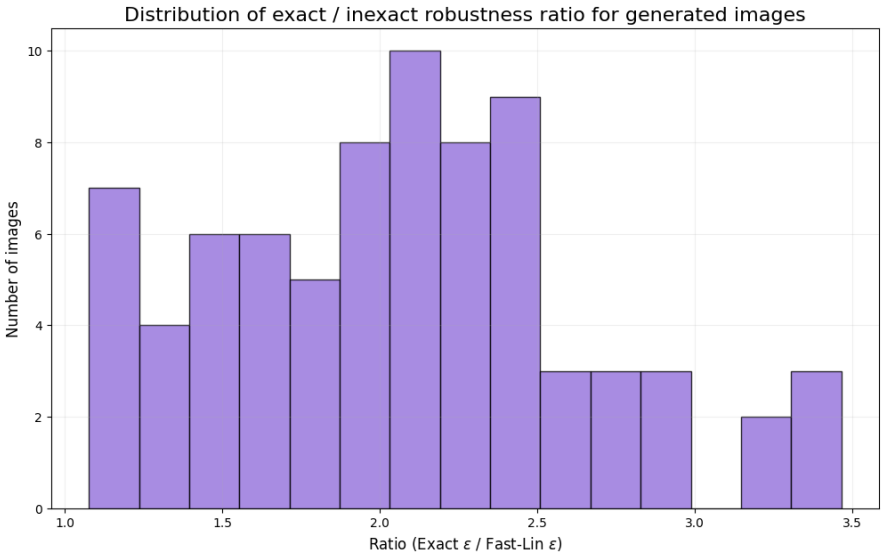

Figure 13: Exact robustness calculated by the MILP divided by certified lower bound calculated by the Fast-Lin method. Only considers generated images.

5.3 Robustness of Generated Images

The robustness of images from the MNIST test set are shown next to the robustness of artificially generated images in Figures 14 and 15. From Figures 14 and 15 we notice that different selection functions yield mostly similar results. In fact, the main contributor to the robustness of the generated images is the class of the generated digit. The threes generate especially robust results, around six times that of the MNIST baseline.

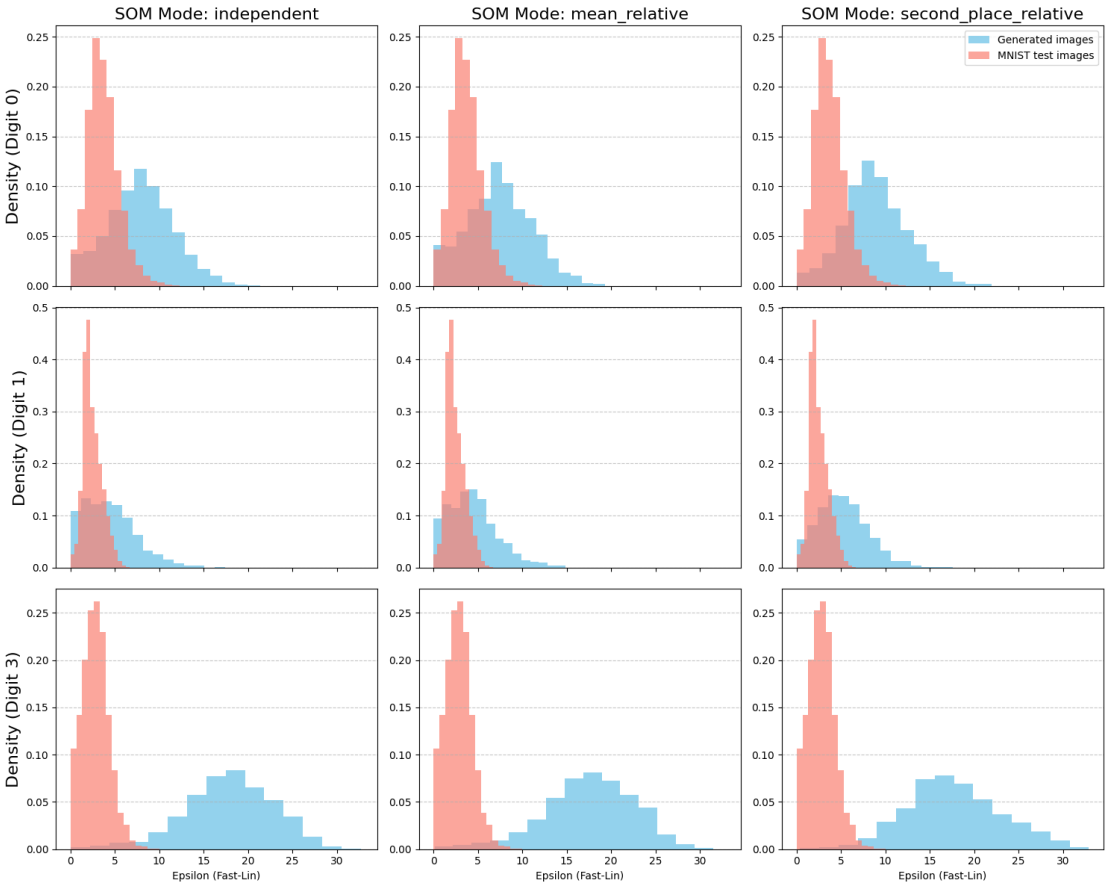

Figure 14: Certified lower robustness bounds calculated by the Fast-Lin method for numbers zero, one, and three. The red parts to the left of the plots are the robustness bounds for images from the MNIST test set. The blue parts further to the right of the plots are the robustness bounds for images generated by selective output maximization. One plot for every investigated combination of digit and selection function.

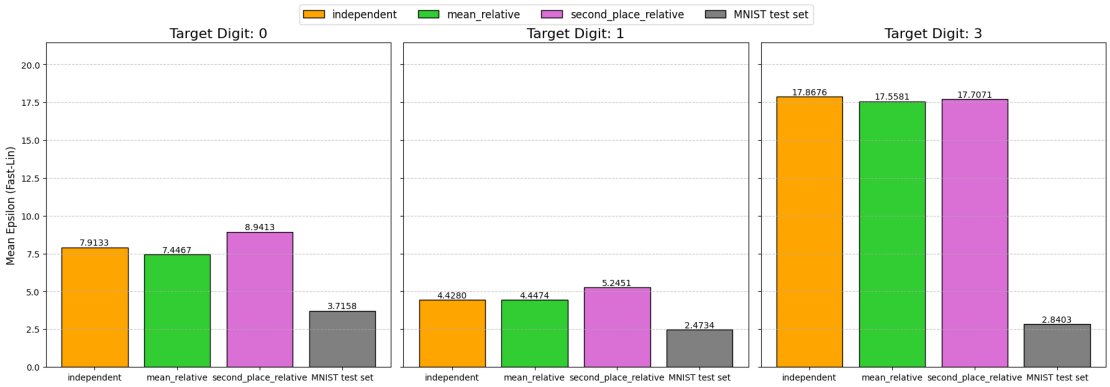

Figure 15: Average certified lower robustness bounds calculated by the Fast-Lin method for each investigated digit and each selection function. The average robustness for the MNIST test set is also included as a baseline.

5.4 Transferability of Generated Images

The robustness transferability results are presented below, both for the images generated by single-models and for those generated by multi-models.

5.4.1 Single-Model Transferability

Figure 16 shows the average robustness of generated images when evaluated on networks that generated the images compared to those networks that did not generate them, depending on which output was maximized to generate the images. Figure 17 shows the average robustness of generated images on networks that generated the images compared to those that did not generate them, depending on which selection function that was used to generate the images. The average robustness of images from the MNIST test set is also provided as a baseline in both figures.

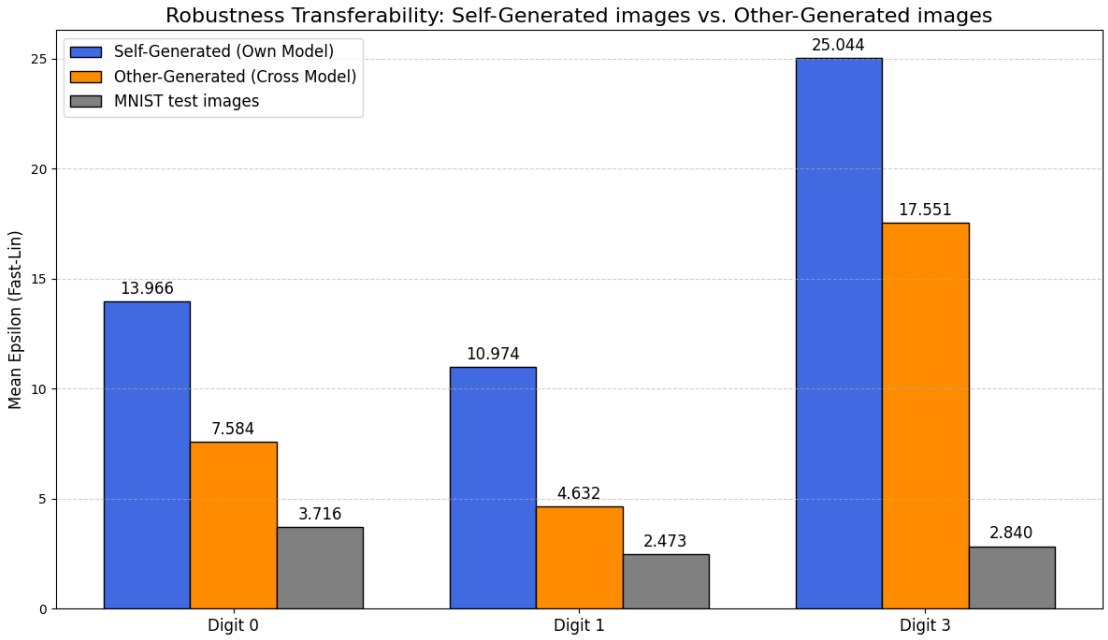

Figure 16: Average robustness lower bounds. Includes the MNIST test set, artificially generated images on their own network, and artificially generated images on other networks.

Figure 16 shows that, while most robust on the networks that generated them, the generated images also transferred to other networks that did not generate them. That is, the generated images are also in general more robust on other networks. Regardless of which digit that was generated, other networks are clearly more robust on the generated image, compared with the MNIST baseline. Figure 17 shows that the type of selection function used to generate the images does not noticeably impact the transferability of the image.

Figure 18 plots the robustness lower bounds of these images against the number of neurons of the generator network, and against the depth of the generator network. Figure 18 shows that the number of hidden neurons of the generator network does not significantly impact the robustness of the generated images. However, the robustness of the generated images seems to somewhat decrease with the number of hidden layers of the generator.

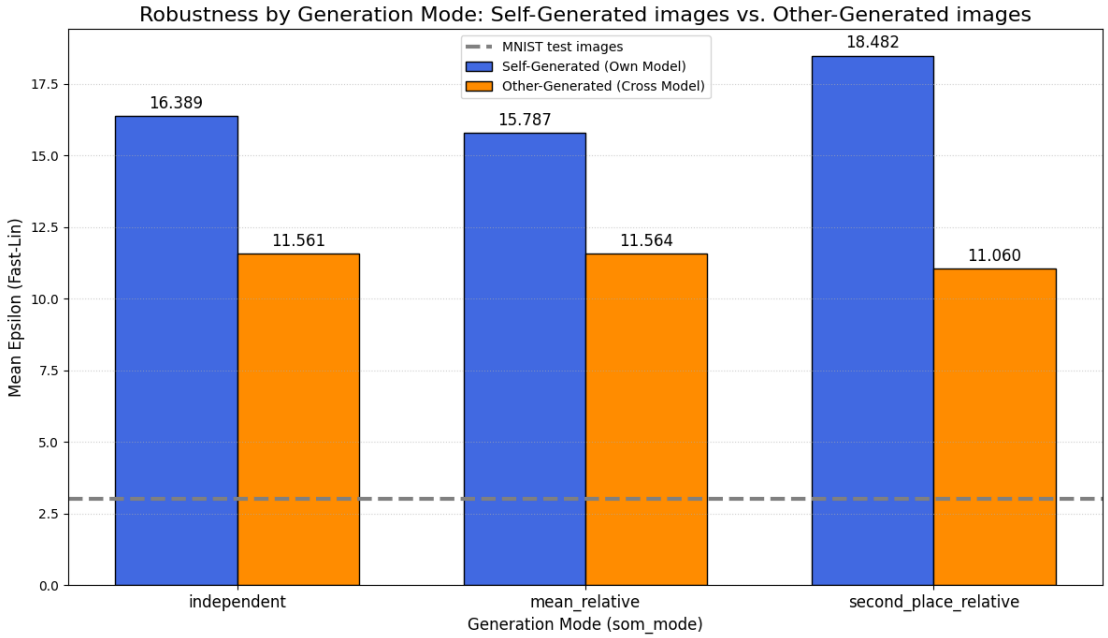

Figure 17: Average robustness lower bounds of artificially generated images on their own network and artificially generated images on other networks. Split by the type of selection function used to generate the images. An MNIST baseline is also provided.

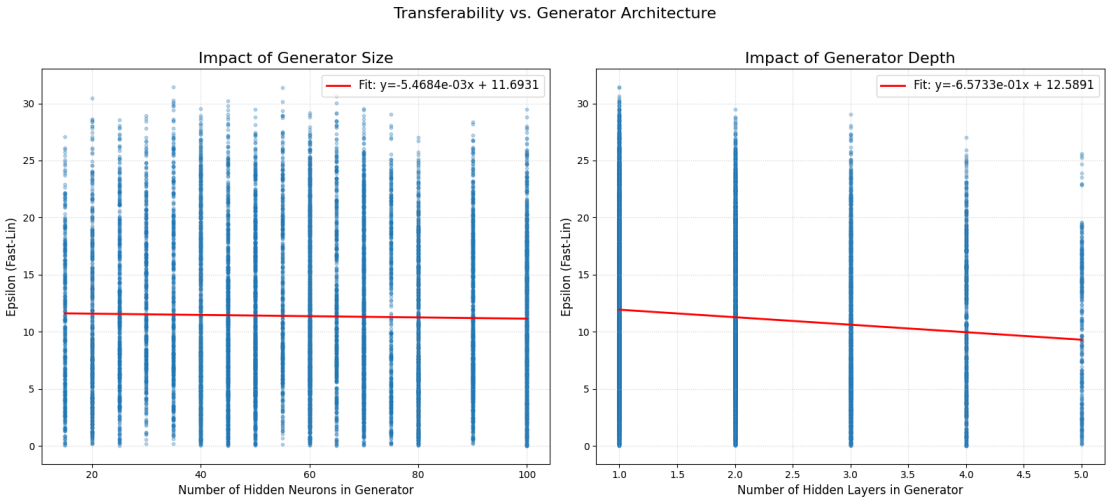

Figure 18: Robustness lower bounds of generated images tested on networks that they were not generated from, plotted against the number of neurons and the network depth of the network they were generated from.

5.4.2 Multi-Model Transferability

Figure 19 shows the robustness transferability results of images generated by multi-models, along with an MNIST baseline for comparison. The multi-model results presented in Figure 19 behave

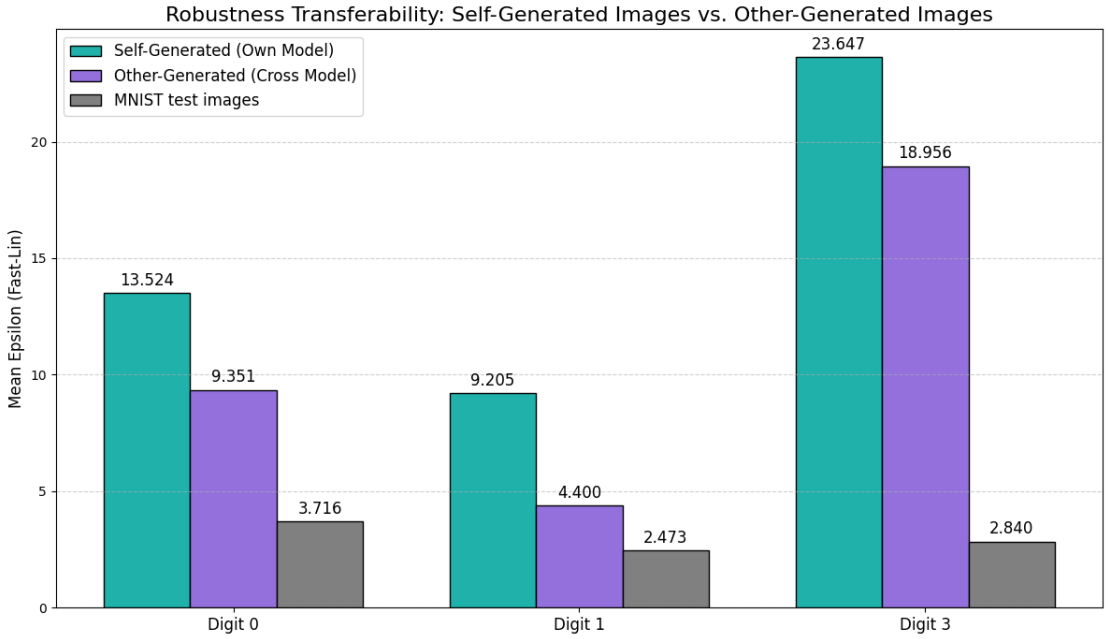

Figure 19: Average robustness lower bounds of artificially generated images on multi-models. Compares robustness of the generated images on networks that generated them, and networks that did not generate them. Also includes an MNIST baseline.

similarly to the single-model results presented in Figure 16. These multi-model images are slightly more transferable compared to the single-model images, while achieving slightly worse robustness on their own networks. Therefore, there does not seem to be a huge benefit in the overall robustness of generated images by using a multi-model generation method.

6 Discussion

The results suggest that Fast-Lin is a good proxy both for the normal MNIST images and also the generated images. The exact to inexact ratio seems to have slightly more variance in the latter case, but the correlation is still strong. We therefore conclude that the network-specific optimization used to generate the images does not, at least to a significant degree, destructively affect the Fast-Lin method's approximation of the exact optimization problem. For this reason, many of the results we provide are computed using only Fast-Lin.

The main result we present is that generating images via selective output maximization indeed does result in images with higher robustness compared to MNIST test images, often substantially higher. We see no particular reason as to why these results would be unique to MNIST images. We also note that the robustness of images is highly dependent on the class, in this case the type of digit. This holds for both the MNIST test images and the artificially generated images. In the MNIST case, digits *zero* are more robust than *ones* and *threes*. The robustness of the generated images of each class expresses an even larger difference between classes. Generated *threes* are the most robust, followed by *zeros* and *ones*. We can only speculate about why this occurs. Perhaps there are simply more features which uniquely characterize *threes* compared to *ones*, the digit *one* is just a line after all. Alternatively, it may be because the flag (short slanted downwards line on the top of some *ones*) makes it easy for the optimization to find a slight perturbation which confuses the networks to classify it as a *seven*, which could look similar.

We observe a statistically significant tendency for deeper networks to be less robust. One possible, although purely speculative, explanation is that the larger depth enables the network to learn more concrete and specific patterns which makes it more prone to small perturbations in those patterns. To make an informal example, imagine that a smaller network only has enough logical space to learn that: "a sample is a *zero* if there is one relatively large area of *zero* valued pixels enclosed by non-*zero* valued pixels". This rule would not be very reliable logic for classification, but in terms of perturbation resilience it would. Another possible explanation is that the deeper networks are simply over-fitted and so the difference in robustness is in line with non-training data classification performance. We deem that a deeper investigation into the relationship between overfitting and robustness would complement the results provided in this report well. It is also worth remembering that the Fast-Lin results are only a proxy for the exact results and the precision of Fast-Lin could degrade as the number of layers increases. The extent to which this influences the results has not been thoroughly investigated in this report.

As expected, the robustness of a generated image is larger on the network which generated the image than on those networks that did not. This confirms and measures the relationship between maximizing robustness and maximizing the confidence-related objectives that the selective output maximization does. If the relationship was weaker, then we would not expect the network that generated the image to consistently be more robust on the generated image. The robustness transferability is also consistently higher than the MNIST baseline. One interpretation of this is that there could be something fundamentally more robust about the generated images. Alternatively, it might also be the case that the generated image improves robustness directly only on the network which generated it, and then that robustness transfers well to networks which are similar in architecture to the one that originally generated the image. Quite likely, it is a combination of both. Exploring this further would be a natural continuation of this paper.

Our results show that selective output maximization produces less robust images from deeper

networks. This could again be speculatively attributed to the deeper networks being trained to recognize slightly more specific and less generalizable patterns that it uses to characterize each class. Generating images from that logic would then result in images whose characteristic features do not align with those of other networks classification logic. If indeed the poorer robustness of the generated images of a network does stem from the complexity/generalizability of the networks classification logic, then it could conceivably serve as a metric thereof. This is but a speculatively suggestion and does not directly pertain to the results of the report. Intuitively, a network with very complex classification logic would not lend itself well to summarizing all that into a single "most representative" input. Meanwhile, a network which has sufficient but very simplistic classification logic would be more likely to classify based on more general simple patterns and therefore generate images with more transferable patterns that other networks are more likely to have internalized.

Consistently, the type of selection function that samples were generated by does not seem to make much difference, neither in terms of robustness nor visual appearance. Both the mean relative and the independent selection functions perform almost identically, whilst the difference the second place relative selection function demonstrates mainly results in a slightly larger transferability gap. The larger gap could be explained by the increased complexity of the selection function simply taking into account more of the network logic, thus generating images slightly more robust on itself but also slightly less transferable.

There is not a huge difference in the robustness between images generated by multi-models compared to images generated by single-models. The results hint at the multi-model images being slightly more transferable to other networks, but also slightly less robust on their own networks. Sadly, the computational constraints of solving neural networks exactly are especially apparent for the multi-model. Solving two networks on average doubles the amount of hidden neurons. This means that only smaller networks are feasible. We may therefore only draw limited conclusions from these multi-model results.

All the experiments are conducted in the ℓ_1 norm. Extending the analysis to other norms could potentially be an interesting direction for future work.

7 Conclusion

Selective output maximization generates more robust images compared to those in the test set. The average robustness of each digit differs: for test images the *zeros* are the most robust digit, for generated images the *threes* are the most robust digit. The robustness of the generated images mostly depends on which digit that is generated and not which specific selection function that is used for the image generation.

The generated images perform most robustly on the network which generated them. This robustness, which is higher than the test-set baseline, does also to varying degrees transfer to other networks as well. We also find that more layers in the generating network on average produces less robust images than those generated from more shallow networks. The robustness transferability seems to be somewhat increased by using multi-model generation, the computational constraints however severely limit the extent to which we can conduct this investigation.

Limitations include both the dataset and the network architectures. Since the obtained results only consider the MNIST dataset and similar neural networks, it could be the case that there are differences for more complex models and datasets. In the future, one could therefore investigate whether the robustness qualities remain for different datasets and more varied neural networks. As briefly mentioned, further research using selective output maximization to study the relationship between robustness and overfitting could also potentially yield interesting results.

References

[1] Abien Fred Agarap. *Deep Learning using Rectified Linear Units (ReLU)*. 2026. arXiv: 1803.08375 [cs.NE]. URL: https://arxiv.org/abs/1803.08375.

[2] Dumitru Erhan et al. *Visualizing Higher-Layer Features of a Deep Network*. Tech. rep. Technical Report 1341. University of Montreal, June 2009. URL: https://papers.baulab.info/papers/also/Erhan-2009.pdf.

[3] Matteo Fischetti and Jason Jo. "Deep neural networks and mixed integer linear optimization". In: *Constraints* 23.3 (July 2018), pp. 296–309. ISSN: 1383-7133. DOI: 10.1007/s10601-018-9285-6. URL: https://link.springer.com/article/10.1007/s10601-018-9285-6.

[4] Python Software Foundation. *Python Documentation (3.11)*. Accessed: 2026-05-27. URL: https://docs.python.org/3.11/.

[5] PyTorch Foundation. *PyTorch Documentation (2.10))*. Accessed: 2026-05-27. URL: https://docs.pytorch.org/docs/2.10/index.html.

[6] Johannes Geier and Patrizia Heinl. "Towards a Metric to Assess Neural Network Resilience Against Adversarial Samples". In: *Availability, Reliability and Security*. Ed. by Florian Skopik, Vincent Naessens, and Bjorn De Sutter. Cham: Springer Nature Switzerland, 2025, pp. 272–290. ISBN: 978-3-032-00642-4.

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[8] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572 [stat.ML]. URL: https://arxiv.org/abs/1412.6572.

[9] LLC Gurobi Optimization. *Gurobi Documentation (13.0)*. Accessed: 2026-05-27. URL: https://docs.gurobi.com/13.0/.

[10] Joey Huchette et al. *When Deep Learning Meets Polyhedral Theory: A Survey*. 2025. arXiv: 2305.00241 [math.OC]. URL: https://arxiv.org/abs/2305.00241.

[11] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[12] Jon Kleinberg and Éva Tardos. *Algorithm design*. eng. First edition, Pearson new international edition. Always Learning. Harlow, England: Pearson, 2014. ISBN: 1-292-03704-0.

[13] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. *Adversarial examples in the physical world*. 2017. arXiv: 1607.02533 [cs.CV]. URL: https://arxiv.org/abs/1607.02533.

[14] Y. Lecun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.

[15] Mark Huasong Meng et al. *Adversarial Robustness of Deep Neural Networks: A Survey from a Formal Verification Perspective*. 2022. DOI: 10.48550/arXiv.2206.12227. arXiv: 2206.12227 [cs.LG]. URL: https://arxiv.org/abs/2206.12227.

[16] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. 2014. arXiv: 1312.6034 [cs.CV]. URL: https://arxiv.org/abs/1312.6034.

[17] Aman Sinha et al. *Certifying Some Distributional Robustness with Principled Adversarial Training*. 2020. arXiv: 1710.10571 [stat.ML]. URL: https://arxiv.org/abs/1710.10571.

[18] Christian Szegedy et al. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199 [cs.CV]. URL: https://arxiv.org/abs/1312.6199.

[19] Juan Pablo Vielma. "Mixed Integer Linear Programming Formulation Techniques". In: *SIAM Review* 57.1 (2015), pp. 3–57. DOI: 10.1137/130915303. eprint: https://doi.org/10.1137/130915303. URL: https://doi.org/10.1137/130915303.

[20] Lily Weng et al. "Towards Fast Computation of Certified Robustness for ReLU Networks". In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, July 2018, pp. 5276–5285. URL: https://proceedings.mlr.press/v80/weng18a.html.

[21] H. Paul Williams. *Model Building in Mathematical Programming*. 5th ed. Chichester, UK: Wiley, 2013.

A Appendix

We add some results that we deem interesting enough to include in the report, but not quite relevant enough to fit in the main part.

A.1 Alternative Image Generation Procedures

We want to compare selective output maximization to some alternative ways of generating images through extracting the essence of a class. A simple way of doing so is by averaging all the test images of some class and then seeing if the resulting image is more robust than normal test images. We also try to average the generated samples to see if that would produce an image with perhaps more transferability and therefore also general robustness. The results are illustrated Figure 20 below. There is no consistent statistically significant difference between the average

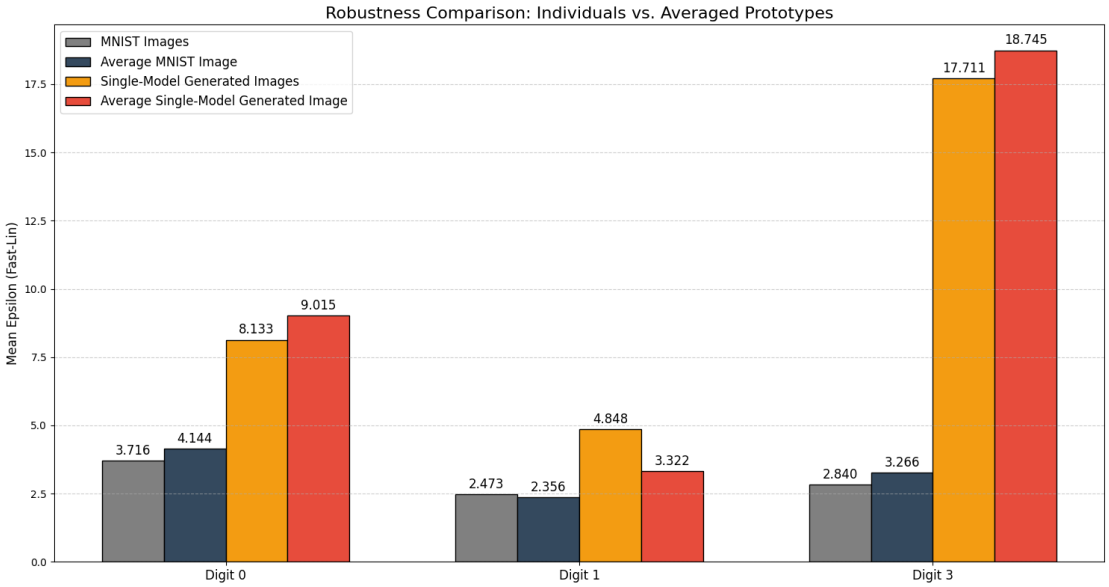

Figure 20: Robustness lower bounds of averaged images. Both an average over the MNIST test set and an average over the artificially generated images are compared.

robustness of the averaged images and the average robustness of the images which were averaged.

An alternative way of generating images, in a spirit similar to the averaging but more advanced, is through PCA. We will not explain what PCA is, but we present the results in Figure 21. There is very little improvement in the robustness of these images.

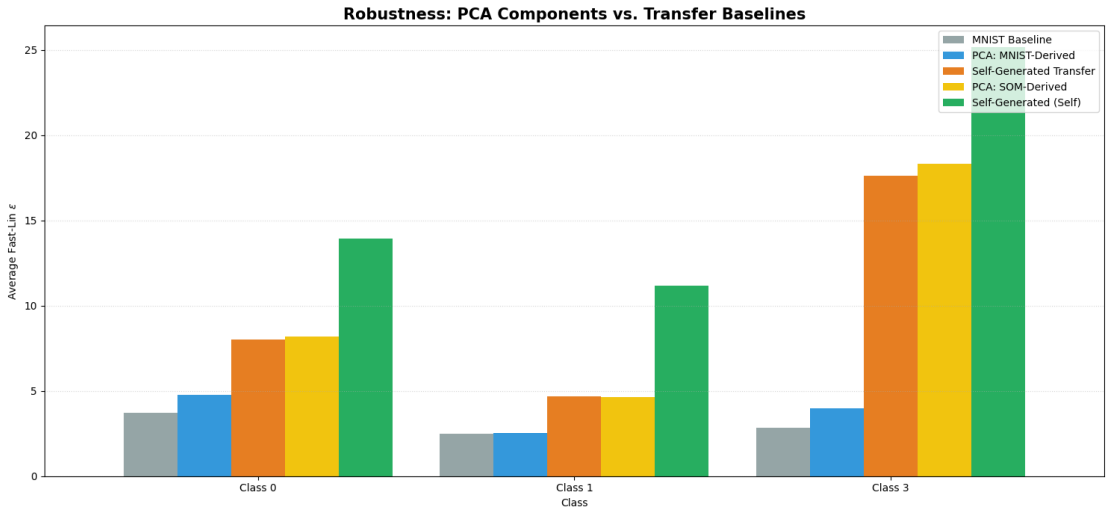

Figure 21: PCA generation comparison.

To summarize the above results and put them in context, we provide the following Figure 22, which shows the average robustness of different images on different types of networks.

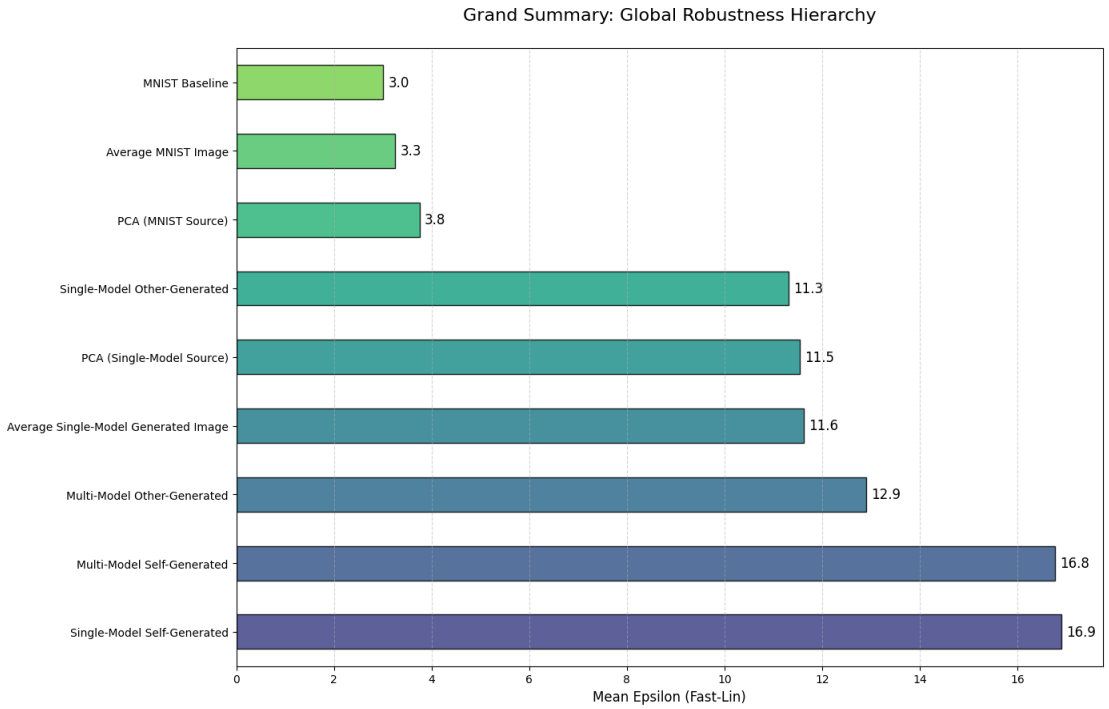

Figure 22: Comparison of robustness of all tested generation methods.

A.2 The Most Robust Samples

The on average most robust sample are shown in Figure 23 below, as ranked by transferred robustness, and where misclassifications are treated as having zero robustness. The heat map corresponds to the robustness results when the images are tested on the network corresponding to each cell.

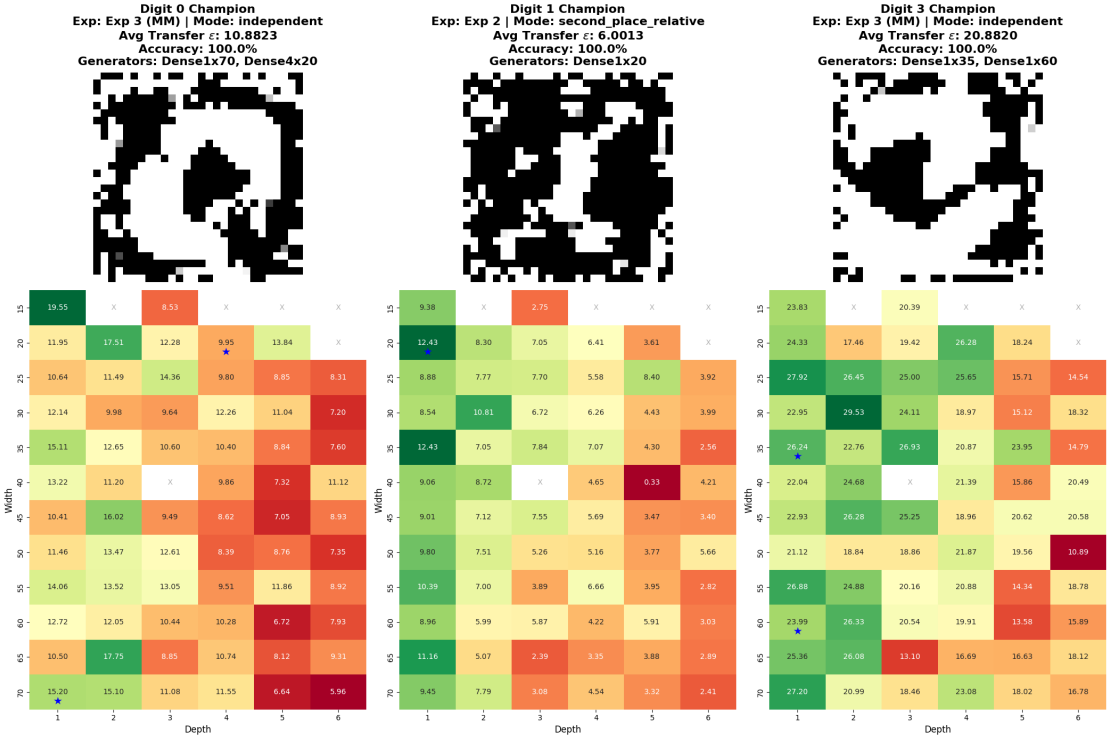

Figure 23: Most robust samples.

A.3 Gallery

Figure 24 below is a quasi-random selection of generated images, perhaps helpful for reader understanding or curiosity.

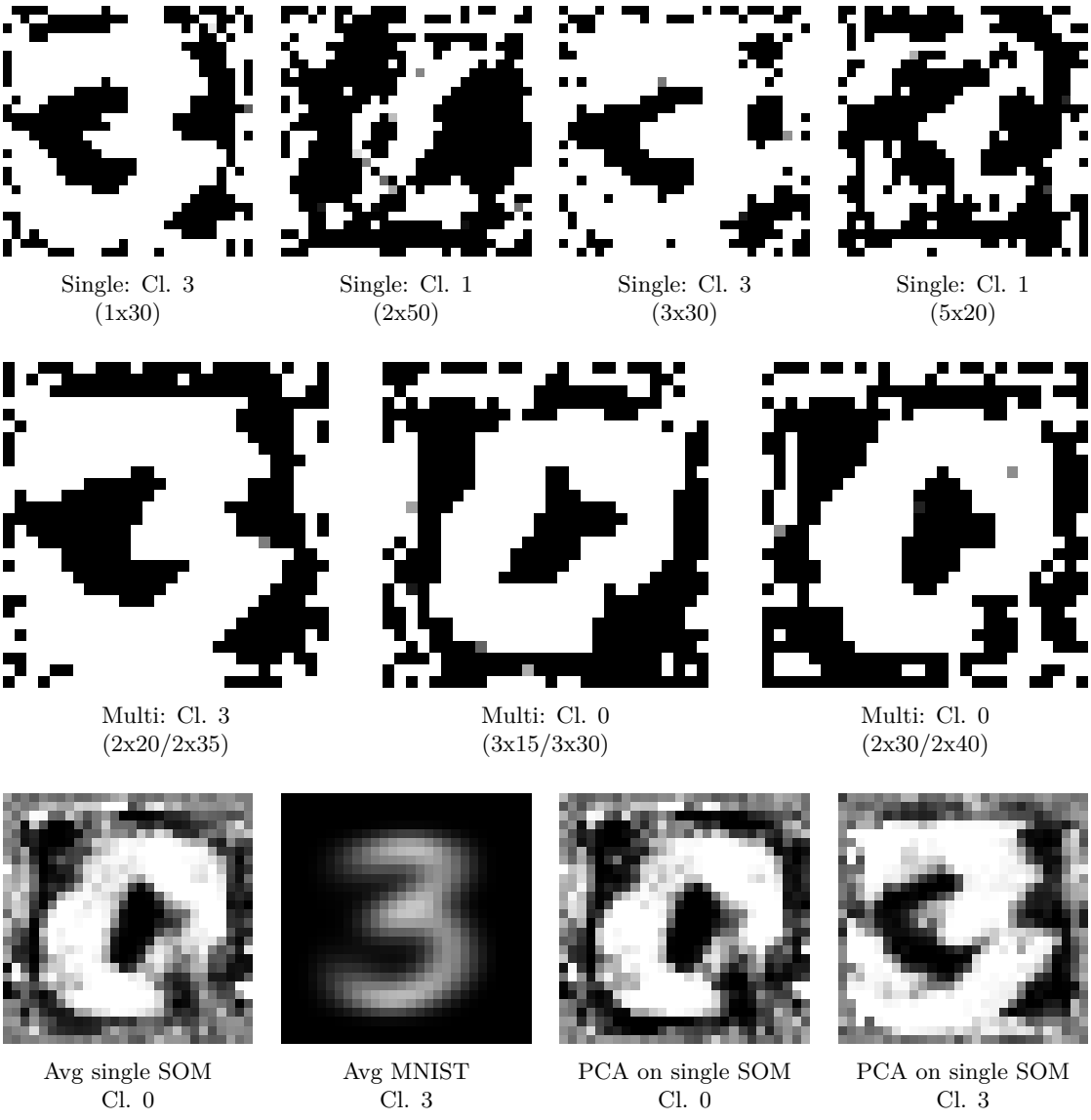

Figure 24: Random selection of generated samples.

A.4 Accuracy of Trained Neural Networks

Width \ Layers	1	2	3	4	5	6
10	~~98.1 / 97.7 / 89.8~~	~~97.8 / 97.4 / 87.8~~	~~98.0 / 97.5 / 86.7~~	~~92.1 / 95.9 / 88.1~~	~~94.7 / 95.9 / 84.1~~	~~95.5 / 94.3 / 78.7~~
15	97.8 / 96.2 / 90.6	~~98.3 / 96.8 / 89.9~~	97.4 / 98.0 / 92.7	~~96.8 / 98.2 / 87.3~~	~~93.3 / 95.7 / 87.4~~	~~97.2 / 95.1 / 83.9~~
20	98.7 / 97.6 / 90.0	98.1 / 98.3 / 92.8	98.0 / 98.1 / 90.2	98.5 / 97.7 / 92.2	98.2 / 96.5 / 92.1	~~96.1 / 96.8 / 85.9~~
25	98.4 / 98.4 / 92.3	97.3 / 97.5 / 93.8	95.0 / 98.5 / 92.7	96.2 / 98.5 / 93.5	98.4 / 97.0 / 90.6	97.0 / 97.8 / 92.9
30	98.4 / 98.0 / 91.8	99.0 / 98.3 / 91.7	97.8 / 96.5 / 94.8	97.6 / 98.2 / 93.0	99.3 / 96.9 / 93.9	96.1 / 96.2 / 92.9
35	98.4 / 97.8 / 92.1	98.4 / 98.2 / 92.5	97.0 / 98.7 / 92.4	97.7 / 97.4 / 92.6	98.2 / 98.2 / 93.4	98.0 / 98.2 / 90.7
40	98.6 / 97.9 / 94.6	98.9 / 98.0 / 93.7	~~98.4 / 98.8 / 89.2~~	97.9 / 98.2 / 95.8	97.9 / 98.3 / 91.9	98.3 / 99.3 / 95.0
45	96.9 / 98.1 / 90.7	97.9 / 98.7 / 94.5	98.4 / 98.5 / 96.5	97.0 / 98.1 / 93.1	97.9 / 97.4 / 94.7	97.8 / 96.3 / 96.3
50	98.7 / 98.6 / 95.3	98.1 / 98.7 / 93.8	98.4 / 98.6 / 93.9	96.9 / 99.0 / 95.5	98.2 / 98.9 / 93.8	98.8 / 98.9 / 95.4
55	98.1 / 98.2 / 92.1	98.9 / 98.7 / 96.5	98.6 / 98.7 / 94.7	98.3 / 98.6 / 96.0	95.5 / 98.9 / 95.2	97.4 / 97.8 / 93.5
60	98.6 / 98.3 / 93.5	97.3 / 98.1 / 96.2	99.1 / 98.4 / 93.4	97.6 / 98.3 / 93.6	97.9 / 98.8 / 96.1	96.9 / 98.7 / 95.1
65	98.6 / 98.4 / 94.8	98.8 / 99.2 / 93.7	98.1 / 98.1 / 91.3	98.7 / 99.1 / 95.7	98.6 / 99.1 / 94.2	99.1 / 97.5 / 94.1
70	98.6 / 98.7 / 96.3	98.5 / 98.7 / 93.5	98.9 / 97.0 / 94.8	98.6 / 98.9 / 96.9	97.7 / 98.6 / 94.4	97.8 / 98.7 / 95.8

Table 1: The trained neural networks and their accuracies for digits zero, one, and three (0 / 1 / 3). Notice that networks with some accuracy less than 90% are marked by a strikeout and ignored. This is explained in Subsection 4.6.

TRITA –
Stockholm, Sweden 2026

www.kth.se